

WEEK 2 INTERNSHIP REPORT

Data Extraction and Cleaning Methodologies in Healthcare Analytics

Project Title

Predictive Analytics for 30-Day Hospital Readmission Risk Among Diabetic Patients

Virtual Healthcare Data Analytics Internship

Tools: Python, Pandas, NumPy, Google Colab

Student Project Report

GitHub Repository: github.com/bhoomikumari45344-web/diabetes-readmission-prediction

1. Introduction

The main aim of this project is to use healthcare data analytics and machine learning to identify diabetic patients who may be readmitted to the hospital within 30 days after discharge. In this week, I focused on data extraction, data quality checking, cleaning, and basic preprocessing using Python. Data cleaning is important in healthcare analytics because real-world datasets can contain missing values, unknown values, duplicate records, and inconsistent information. If these issues are not handled properly, they can affect later analysis and machine learning results.

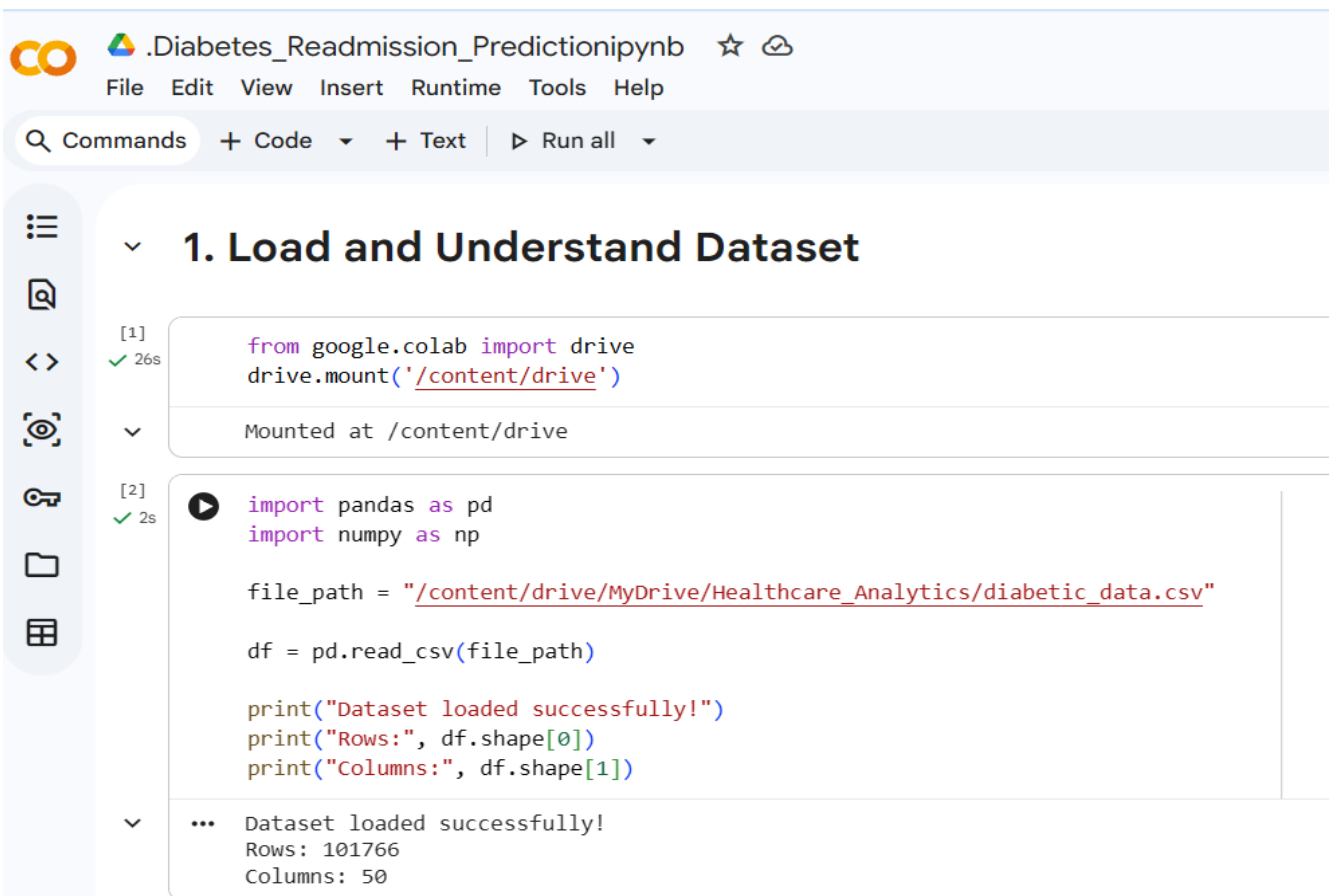
2. Dataset and Data Extraction

I used the Diabetes 130-US Hospitals for Years 1999–2008 dataset. The dataset contains information about diabetic patient hospital encounters. I stored the CSV file in Google Drive and loaded it into Google Colab using Pandas. The dataset loaded successfully with 101,766 rows and 50 columns.

The main code used was:

```
df = pd.read_csv(file_path)
```

After loading, I checked the shape of the dataset to confirm that the file was read correctly.

The image is a screenshot of a Google Colab notebook. The top bar shows the Colab logo and the notebook title ".Diabetes_Readmission_Prediction.ipynb". Below the title is a menu bar with options: File, Edit, View, Insert, Runtime, Tools, and Help. A search bar labeled "Commands" is on the left. The main area of the notebook is titled "1. Load and Understand Dataset". It contains two code cells. The first cell, labeled [1] and 26s, contains the code:

```
from google.colab import drive
drive.mount('/content/drive')
```

 Below the code is the output: "Mounted at /content/drive". The second cell, labeled [2] and 2s, contains the code:

```
import pandas as pd
import numpy as np

file_path = "/content/drive/MyDrive/Healthcare_Analytics/diabetic_data.csv"

df = pd.read_csv(file_path)

print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

 Below the code is the output: "... Dataset loaded successfully! Rows: 101766 Columns: 50". On the left side of the notebook, there is a sidebar with icons for file management, search, and other functions.

Figure 1. Dataset loading output

3. Creating a Cleaning Copy

To avoid directly changing the original dataset, I created a separate copy called `clean_df`. This allowed me to perform all cleaning operations while keeping the original data available for reference.

```
Code used: clean_df = df.copy()
```

4. Identifying Missing and Unknown Values

The dataset used the symbol '?' for some unknown values. I checked both '?' values and actual NaN values. The check showed missing or unknown information in columns such as race, weight, payer_code, medical_specialty, diag_1, diag_2, diag_3, max_glu_serum, and A1Cresult.

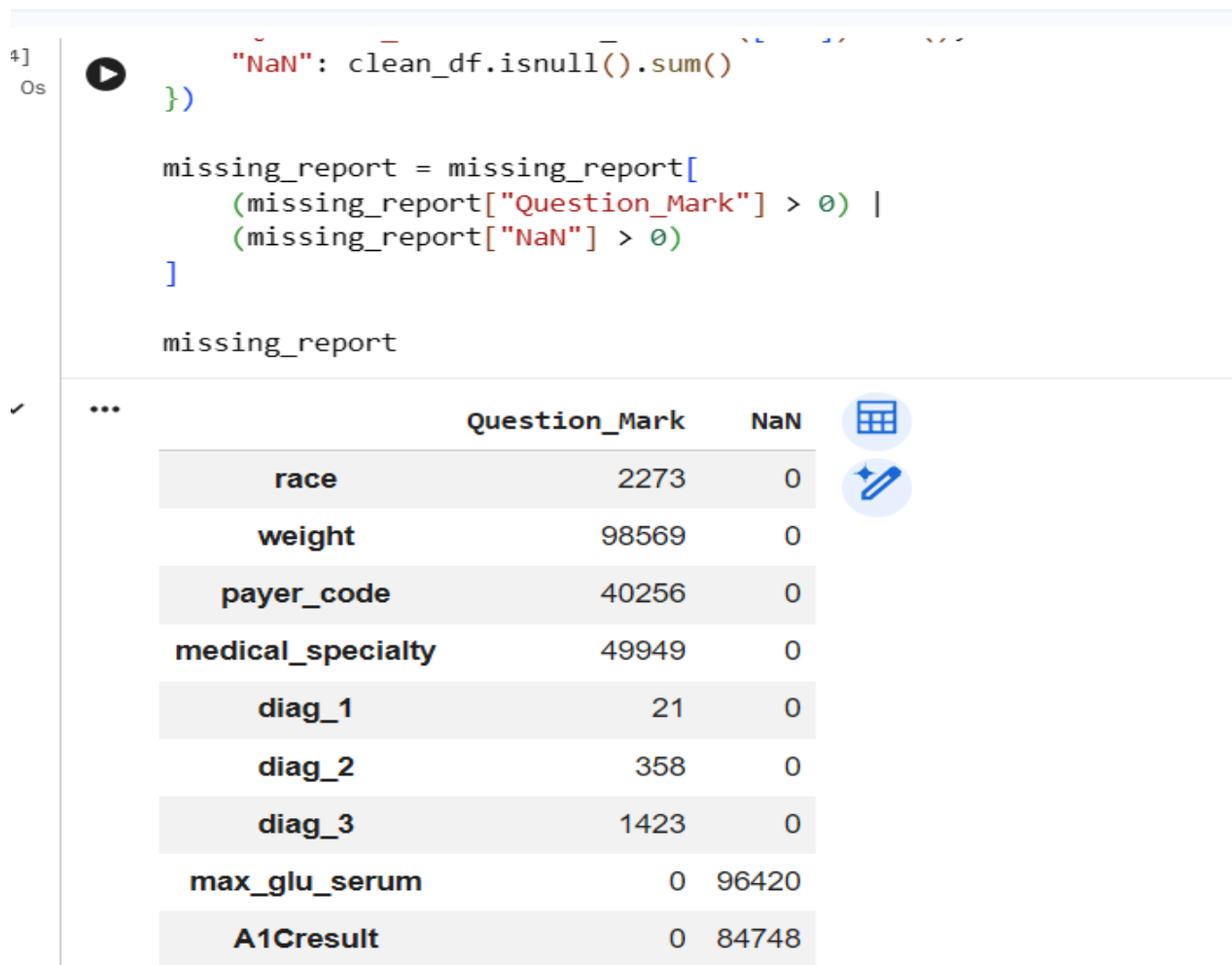


Figure 2. Missing-value analysis

5. Converting Unknown Values

I converted the '?' symbol into NaN using Pandas and NumPy. This made the missing values consistent and easier to handle during cleaning.

```
Code used: clean_df = clean_df.replace('?', np.nan)
```

6. Handling Highly Missing Columns

After checking the missing values, I removed five columns that had a very large amount of missing information: weight, max_glu_serum, A1Cresult, medical_specialty, and payer_code. For example, weight had 98,569 unknown values, while max_glu_serum and A1Cresult had 96,420 and 84,748 NaN values respectively. Instead of making a large number of assumptions through imputation, these highly incomplete columns were removed for this project.

```
1
0s high_missing_cols = [
    "weight",
    "max_glu_serum",
    "A1Cresult",
    "medical_specialty",
    "payer_code"
]

clean_df = clean_df.drop(columns=high_missing_cols)

print("Removed columns:", high_missing_cols)
print("New shape:", clean_df.shape)

Removed columns: ['weight', 'max_glu_serum', 'A1Cresult', 'medical_specialty', 'payer_code']
New shape: (101766, 45)

1
0s categorical_missing_cols = [
    "race",
    "diag_1",
    "diag_2",
    "diag_3"
]

clean_df[categorical_missing_cols] = clean_df[categorical_missing_cols].fillna("Unknown")

print("Remaining missing values:")
print(clean_df.isnull().sum().sum())

... Remaining missing values:
0
```

Figure 3. High-missing columns removed and remaining missing values checked

7. Handling Remaining Missing Values

The remaining missing values were in race and diagnosis-related columns. Instead of removing complete patient records, I filled missing values in race, diag_1, diag_2, and diag_3 with the label 'Unknown'. After this step, the total number of remaining missing values was 0.

8. Duplicate Check

Duplicate rows can affect analysis and model results, so I checked the cleaned dataset for exact duplicate rows. The result was 0 duplicate rows. Therefore, no rows needed to be removed because of duplication.

9. Preparing the 30-Day Readmission Target

The original readmitted column contains three categories: NO (54,864), >30 (35,545), and <30 (11,357). Because the project focuses on readmission within 30 days, I created a new binary target called readmitted_30. The value <30 was mapped to 1, while >30 and NO were mapped to 0. The original readmitted column was kept unchanged.

The resulting target distribution was 90,409 records in class 0 and 11,357 records in class 1.

10. Identifier Columns

The dataset contains encounter_id and patient_nbr. These are identifiers and do not represent useful clinical characteristics. They are retained in the cleaned dataset for reference, but they will be excluded from the machine learning feature set later so that the model does not learn meaningless ID patterns.

11. Final Data Quality Check

A final validation was performed after the cleaning steps. The final cleaned dataset contained 101,766 rows and 46 columns. There were 0 remaining missing values and 0 duplicate rows. The target distribution was 90,409 for class 0 and 11,357 for class 1.

```
print("Final cleaned dataset shape:", clean_df.shape)

print("\nRemaining missing values:")
print(clean_df.isnull().sum().sum())

print("\nDuplicate rows:")
print(clean_df.duplicated().sum())

print("\nTarget distribution:")
print(clean_df["readmitted_30"].value_counts())
```

```
... Final cleaned dataset shape: (101766, 46)

Remaining missing values:
0

Duplicate rows:
0

Target distribution:
readmitted_30
0    90409
1    11357
Name: count, dtype: int64
```

Figure 4. Final cleaned dataset validation and target distribution

12. Tools and Libraries Used

Python: Main programming language for data extraction and preprocessing.

Pandas: Used for loading the CSV file, checking missing values, removing columns, checking duplicates, and creating the target variable.

NumPy: Used for handling missing values through NaN.

Google Colab: Used as the coding environment for the practical work.

13. Assumptions and Cleaning Decisions

The cleaning decisions were made based on the amount and type of missing information. Highly incomplete columns were removed instead of applying heavy imputation. Missing categorical values in selected low-missing columns were represented as Unknown. For the prediction task, <30 readmission was treated as the positive class, while >30 and NO were combined into the negative class. Identifier columns will not be used as predictive features. These decisions provide a cleaner dataset for the next stages of exploratory analysis and predictive modelling.

14. Conclusion

In Week 2, I successfully extracted and cleaned the diabetes healthcare dataset using Python. The original dataset had 101,766 records and 50 columns. I identified missing and unknown values, converted '?' to NaN, removed five highly incomplete columns, handled the remaining selected missing categorical values, and checked for duplicates. I also created the readmitted_30 target for the 30-day readmission prediction problem. The final dataset had 101,766 rows and 46 columns, with 0 missing values and 0 duplicate rows. This cleaned dataset is now ready for exploratory data analysis and visualization in Week 3.

GitHub Repository: <https://github.com/bhoomikumari45344-web/diabetes-readmission-prediction>